

P395: Guide 3:

1. Start up a new notebook and title it 'Guide3_your_name'. Remember to put in helpful headings and sub-headings to keep your notebook organized. We won't be using interactive plots this week so you can load

```
import matplotlib.pyplot as plt
import numpy as np
```

Task 1: Numerical differentiation

The definition of a derivative is

$$\frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

On a computer it is not possible to take this limit, so the above must be evaluated at some finite value for h . Using a Taylor series for $f(x+h)$, we get an approximation for the derivative that has an error proportional to h

$$\frac{df}{dx} = \frac{f(x+h) - f(x)}{h} + O(h)$$

1. Using the approximate *forward-difference* formula above, calculate the derivative of $f(x) = x^{0.8}$ at $x = 1$, using a (base-10) logarithmic range of h from $1e-12$ to $1e-01$.
2. Analysis: Use 1st-year calculus to calculate the exact value of $f'(1)$. Put your answer in a text cell.
3. Plot the absolute error of your approximate values as a function of h on log-log scale.
4. Comment on what you see. (Note: you will likely see your error going up again at really small h . What do you think this is from? Hint: Recall rounding error from an Guide 2; if there is a constant error in evaluating f , how will this affect the error in the derivative?)
5. Analysis: Using Taylor expansions for $f(x+h)$ and $f(x-h)$, show that a better approximation to the derivative is the *central-difference* formula,

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + O(h^2)$$

where the error is now proportional to h^2 . Put your derivation in a text cell.

6. Carry out the same calculation as in (1) and (3), but now using the central-difference formula in (5). Comment on what you see in a text cell.

Further approximations can be made that reduce the error to $O(h^4)$ using more Taylor expansions and more function evaluations at other points. Such methods are only truly useful if the function f is calculable at specific x . In practice, we are more likely to have f evaluated only at certain points beyond our control.

If you have a function evaluated at set points, you can use `np.diff` to calculate the derivative using the forward-difference formula, or my preferred method is to use `np.gradient` which uses the central-difference formula.

Background reading: [Solutions to ordinary differential equations](#)

In many areas of physics, we encounter ordinary differential equations (ODEs) which describe how various physical variables change their state as a function of the system. Some of these differential equations may be higher-order ODEs involving higher-order derivatives (e.g., Newton's 2nd law involves the 2nd derivative of position with respect to time). However, all higher-order ODEs can be represented as a system of coupled first-order ODEs that only involve first-order derivatives. Thus, in general, the ODEs that we encounter in physics can be represented as the following set of equations,

$$\frac{dx_i}{dt} = f_i(t, x_1, x_2, \dots, x_n) \quad (1)$$

Here $i = 1, \dots, n$, and we know the functions f_i . For example, consider Newton's 2nd law:

$$m \frac{d^2x}{dt^2} = F(t, x, v)$$

where x is the position of the particle and F is the force acting on it. This 2nd-order differential equation can be written as two coupled 1st-order differential equations,

$$\begin{aligned} \frac{dx}{dt} &= v(t) \\ m \frac{dv}{dt} &= F(t, x, v) \end{aligned} \quad (2)$$

Here we track both the position x and velocity v as a function of time. For example, an object in free fall would have equations,

$$\begin{aligned} \frac{dx}{dt} &= v \\ \frac{dv}{dt} &= -g \end{aligned} \quad (3)$$

In carrying out any integration, it is necessary to specify either initial conditions or boundary values. We will focus predominantly on initial-condition problems where the values of x_i at some initial point are given. The general idea with integrating ODEs is to pick some finite step size $\Delta t = h$ and then starting from the initial values, use numerical approximations to the derivatives in the ODEs to update the x 's. As we will see, this forward iteration using truncated approximations to reality can accumulate error, potentially leading to divergences.

Task 2: Exploring numerical methods for solving ODEs

Euler method

The simplest method is the Euler method which, for a set of 1st-order differential equations as in Eq. (1), finds the solution at the next step $t_{i+1} = t_i + h$ as

$$x_{i+1} = x_i + h f(t_i, x_i)$$

In our example above for free fall (Eq. 3), applying Euler's method would give the following update rules for the velocity and position

$$x_{i+1} = x_i + h v_i$$

$$v_{i+1} = v_i - h g$$

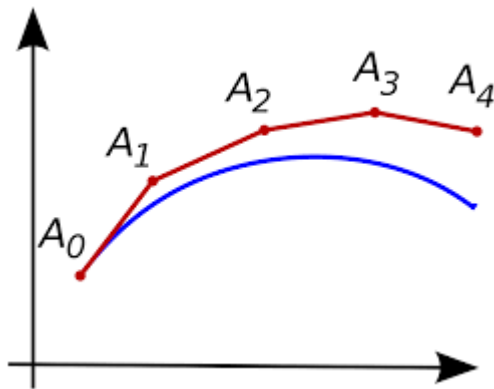
Here $h = \Delta t$ is our increment in time.

Consider the following kinematics problem: A ball starts at a height of 1 m and is thrown up in the air with an initial velocity of 5 m/s.

1. Plot its exact height as a function of time from $t = 0$ to 1 s using the 1D kinematics solution that you remember from 1st-year physics.
2. Use the Euler method to find the position and velocity of the object from $t = 0$ to 1 s, using $h = \Delta t = 0.01$ s.
3. Plot your solution for the position from the Euler method alongside the exact solution from 1st-year physics. What do you notice?

Better Methods: n th-order Runge-Kutta

As you can see, the Euler method accumulates error quickly as can be seen in the figure below.



The actual solution is shown in blue, but Euler just makes a linear prediction to the next timepoint based on the past value. Its error at each step goes as $O(h^2)$, and for many time steps this can accumulate into a large error from the actual solution. Clearly improvements can be made.

The idea is to use a mid-way step to get a better estimate of how the solution is changing. The first approach is what is known as the mid-point or 2nd-order Runge-Kutta method and gives a local error of $O(h^3)$. The update rules are

$$k_1 = h f(t_i, y_i)$$

$$k_2 = h f\left(t_i + \frac{1}{2}h, y_i + k_1\right)$$

$$y_{i+1} = y_i + k_2$$

By symmetrizing and using the value of the derivative at the midpoint, this cancels the term in the approximation that is 1st order in h , leaving a method that is accurate to 2nd order and hence has error at the 3rd order. [In general, an ODE method is of order n if the error is $O(h^{n+1})$.]

The most common ODE solver is a 4th-order Runge-Kutta method. It uses more function evaluations (four of them) to make better estimates for updating. Its steps are

$$k_1 = h f(t_i, y_i)$$

$$k_2 = h f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}k_1\right)$$

$$k_3 = h f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}k_2\right)$$

$$k_4 = h f(t_i + h, y_i + k_3)$$

$$y_{i+1} = y_i + \frac{1}{6}k_1 + \frac{1}{3}k_2 + \frac{1}{3}k_3 + \frac{1}{6}k_4$$

Fortunately, Python implements this method for us (it's the default ODE solver in `scipy`).

4. Look up and read the documentation on `scipy.integrate.solve_ivp`

Let's use `solve_ivp` to solve the free-fall problem.

5. Write a function that returns the derivatives of position and velocity at time t for free fall. Based on the `solve_ivp` documentation it should be a function $f(t, y)$ where y is an array containing $[x(t), v(t)]$.
6. Call `solve_ivp` to solve the free-fall problem from $t = 0$ s to 1 s using initial conditions $[x(0), v(0)] = [1 \text{ m}, 5 \text{ m/s}]$ and a time span of $(0, 1\text{s})$, with no other arguments passed.
7. Plot your solution for position versus time. You should notice something a bit surprising (or unexpected).

Adaptive step-size methods

From your plot you might be asking, "what happened to all the timepoints?" Python's built-in ODE solvers use adaptive step sizes. Instead of taking a fixed step size, it allows for variable step sizes that keep the solution to within some truncation-error tolerance. The benefit is that when the solution isn't changing much, it can take bigger steps in time.

Here's a brief aside on adaptive step sizes. The basic idea is to compare the solution from taking two small steps (size h) versus a single bigger step (size $2h$). Take the difference in the solution to get an estimate Δ of the error. For the 4th-order Runge-Kutta method, Δ goes as (step size)⁵ (i.e., 2 steps of size h give an error of $2 h^5$ compared to one step of size h that gives error on the order of h^5). Let's say

we set our tolerance on this error at Δ_0 . So if we take a step h_1 and get an error estimate Δ_1 , then we can estimate the step size that we can take to keep the error below Δ_0 as

$$h_0 = h_1 \left| \frac{\Delta_0}{\Delta_1} \right|^{1/5}$$

(See Numerical Recipes 17.2 for more details.) Thus we can keep adjusting our steps as we integrate forward in time, keeping the error bounded. And that's basically what the Python routines are doing, and why you saw so few time points in your solution.

8. Read up on how to use `solve_ivp` to solve the ODE at a given number of timepoints using `t_eval`.
9. Now use `solve_ivp` to find the solution on a uniform grid of points from (0, 1) s using $h = 0.01$ s, as you had done before for Euler.
10. Plot the numerical solution from (2) vs. the exact solution for the height vs. time on the same plot. It should be much better than what you found for Euler.

Task 3: Applications

1) *Driven damped harmonic oscillator.*

There are many ODEs that we encounter in physics, but let's look at the familiar one of a damped harmonic oscillator with forcing,

$$m \frac{d^2x}{dt^2} + c \frac{dx}{dt} + kx = F$$

Here m is the mass, c is the damping, k is the stiffness and F is some constant external force. This can be rewritten as

$$\frac{d^2x}{dt^2} + 2\gamma\omega_0 \frac{dx}{dt} + \omega_0^2 x = f$$

where the undamped angular frequency is $\omega_0 = \sqrt{k/m}$, the damping ratio is $\gamma = c/(2\sqrt{mk})$, and $f = F/m$.

1. As shown in the background reading for Task 2, convert this 2nd-order ODE into two 1st-order ODEs. Write in a text cell the two 1st-order differential equations that you need to solve.
2. Write a function that returns the derivatives of (x, v) . It should also take γ , ω_0 , and f as additional parameters.
3. Use `solve_ivp` to find the solution from $t = 0$ to 10 s on a uniform grid using a spacing of $\Delta t = 0.01$ s with initial conditions $x(0) = 0$, $v(0) = 0$ and using $\omega_0 = 2$ and $f = 1$ with the following values of the damping ratio

$$\gamma = 0, 0.25, 0.5, 1, 2$$

(NOTE: you can use the syntax

```
sol = solve_ivp(your_derivs, args=(a, b, c, ...), ... )
```

where $(a, b, c, \dots)$ are the additional parameter arguments to your derivatives function.)

4. Plot your solutions at the different values of γ .
5. Look up the various regimes of a damped harmonic oscillator on Wikipedia. Write a comment in a text cell on your observed solutions at the different values of c and which regime each belongs to.

Here you will look at the behaviour of a driven damped harmonic oscillator. Now make your forcing term time dependent, $f(t) = f \cos \omega t$.

6. Use values of $\gamma = 0.2$ with all the other parameters the same as above. Plot your solution for $\omega = 0.5\omega_0, 0.95\omega_0, 1.45\omega_0$. (You might need to use a larger range of times.) Comment in a text cell on what you see.
7. For $\gamma = 0.2, 0.5$, plot the resulting amplitude of the oscillations as a function of the driving frequency ω . Use as a range of ω from say $[0.2\omega_0, 2\omega_0]$. As you make the damping ratio greater, what do you see?

II) *Intro to chaos*

Chaotic systems are deterministic systems (important to note, it doesn't mean they are stochastic) whose trajectories never repeat and are very sensitive to initial conditions. A classic example is a set of equations that were developed by Lorenz in the context of convection due to heating (see Wikipedia for more info). The three equations he derived to describe the physical system can be summarized as

$$\frac{dx}{dt} = s(y - x)$$

$$\frac{dy}{dt} = rx - y - xz$$

$$\frac{dz}{dt} = xy - bz$$

The system of equations has three parameters, s , r , and b , that control its behaviour. With $s = 10$ and $b = 8/3$, the system is chaotic for $r > 24.7$.

8. Let's first look at the system in the non-chaotic regime. Solve the ODEs from $t = 0$ to $t = 8$ with $\Delta t = 0.01$ for a value of $r < 24.7$. Plot solutions for the following initial conditions: $(-10, 0, 0)$, $(10, 0, 0)$, and $(10, 5, 0)$. How many attractors does it appear to have?
9. Now try a value of r in the chaotic regime. Use $t = 0$ to $t = 100$ to get a lot of points. Make a 2d scatter plot of two of the three variables. (Or you can do a 3d plot.) You should see the Lorenz strange attractor: a trajectory that never repeats.

Task 4: *Wrap up*

1. Please provide any feedback on questions that you found difficult to understand because of the wording.
2. Please provide any other comments about this notebook.